Grad-CAM and SmoothGrad from Scratch: Project Report

Nov 13, 2026

1. Introduction

We implemented two saliency methods from scratch in PyTorch, Grad-CAM and SmoothGrad, using only plain autograd and no XAI library such as captum or pytorch-grad-cam. Both explain why a pretrained ResNet-50 predicts a given ImageNet class by highlighting the image regions that drove the prediction.

Beyond the two methods, we added a third (Smooth Grad-CAM, Grad-CAM averaged over noisy inputs), a faithfulness evaluation based on Deletion and Insertion curves, unit tests (40 in total), sanity checks on real images, and ablations over noise level, sample count and Grad-CAM target layer. The final numbers come from a 200-image run on a Kaggle T4 GPU.

Member	Roll number	Contribution
Saumya Singh	24AI10006	SmoothGrad implementation and its 32 unit tests, sanity checks on real images, noise and sample-count ablations, documentation
Ishika Pajankar	24AI10028	Grad-CAM implementation and tests, sanity checks on real images, documentation of results and issues
Yash Atul Dighade	24AI10049	ImageNet subset loading, pretrained ResNet-50 setup, overlay helpers, Deletion/Insertion metrics
Abhishek Kumar	24AI10007	Full GPU evaluation (Kaggle T4, 200 images), noise, sample-count and layer ablations, comparison figure and overlays, README
Aarjav Desai	24AI10022	Implemented the vanilla-gradient and Random baselines, set up the repository (requirements, install steps, data instructions), and wrote this project report

2. Dataset and model

We evaluated on 200 images drawn at random (seeded) from the Imagenette validation set, explained with an ImageNet-pretrained ResNet-50 from torchvision.

Imagenette. Imagenette is a small subset of ImageNet containing ten easily distinguishable ImageNet classes. We used the 160-pixel version (`imagenette2-160`), whose validation folder has the same folder-per-class layout as ImageNet.

Why we used it:

- The images come from ImageNet classes, so the pretrained ResNet-50 can classify them without any retraining.
- It is small enough to download in seconds and run on free hardware (Kaggle T4), while a full ImageNet validation run would be impractical for SmoothGrad, which needs about 50 gradient passes per image.
- Saliency methods have no ground-truth labels to compare against, so the explanation target is the model's own predicted class. This means we need no labels, only images.

Pretrained ResNet-50. The model is loaded with torchvision's ImageNet weights and is always kept in `eval()` mode. Grad-CAM hooks the last convolutional stage (`layer4`). The code also supports VGG16 (`features.29`), and we used random-weight models and synthetic images (`--fake`) for offline smoke tests and unit tests.

3. Libraries used

The project deliberately depends on very little. Both algorithms are written directly on PyTorch autograd, and no explanation library is used.

Library	What we used it for
PyTorch	Tensors, `torch.autograd.grad` for class-score gradients, forward hooks to capture `layer4` activations in Grad-CAM, bilinear upsampling, and a private `torch.Generator` for reproducible SmoothGrad noise
torchvision	The pretrained ResNet-50 (and VGG16) classifier, plus image loading and preprocessing
matplotlib	Heatmap overlays, the method comparison figure, and the noise, sample-count and layer ablation plots

Library	What we used it for
pytest	The 40 unit tests (8 for Grad-CAM, 32 for SmoothGrad), run on random weights so no download is needed
NumPy	Array handling for saliency maps, heatmap overlays and the metric computations (for example correlations between maps and Deletion/Insertion curves)
Pillow	Opening and resizing the input images (JPEG and PNG) before preprocessing, and saving overlays

The code runs on CUDA when available and falls back to the CPU otherwise.

4. Algorithms implemented

Grad-CAM (Selvaraju et al., 2017) weights the channels of a late convolutional layer by how much the class score depends on them. For class c and activations A with K channels:

```
alpha_k = mean over (i, j) of  d y^c / d A^k_ij
CAM     = ReLU( sum over k of  alpha_k * A^k )
```

Here y^c is the pre-softmax score. A forward hook stores the layer output, `torch.autograd.grad` gives the gradients, and the map is upsampled bilinearly to the input size and min-max normalised. The default layer is `layer4` for ResNet.

SmoothGrad (Smilkov et al., 2017) averages the vanilla gradient saliency over N noisy copies of the input:

```
M(x)     = d y^c / d x
M_hat(x) = (1/N) * sum over i = 1..N of  M(x + noise_i)
noise_i  ~ Normal(0, sigma^2),  sigma = noise_level * (x_max - x_min)
```

Colour channels are collapsed with a max (or sum) over absolute values, and the result is min-max normalised. We support three averaging modes: `mean` (the paper's version), `abs` and `square`. Noise is reproducible through a `seed` argument, and an optional percentile clip (display only) stops a few extreme pixels from darkening the whole map.

Smooth Grad-CAM (our extra) is Grad-CAM averaged over noisy inputs, to test whether smoothing helps a method that is already smooth.

Evaluation by Deletion and Insertion (Petsiuk et al., 2018). There is no ground-truth explanation, so we measure faithfulness on the predicted-class probability:

- **Deletion AUC (lower is better):** remove the most salient pixels first and track how fast the probability collapses.
- **Insertion AUC (higher is better):** reveal the most salient pixels first on a blurred image.
- **Random saliency** is reported as a baseline that every useful method should beat.

5. Results

Grad-CAM scored best on Insertion minus Deletion (0.433) and ran about 100 times faster than SmoothGrad. The table below is the final run: 200 Imagenette images, ResNet-50, SmoothGrad with 50 samples and noise 0.15, on a Kaggle T4 GPU.

Method	Deletion ↓	Insertion ↑	Ins − Del ↑	sec / 100 img
Random	0.1119	0.2163	0.1044	0.00
Vanilla gradient	0.0861	0.2884	0.2023	0.76
SmoothGrad (mean)	0.0601	0.4241	0.3641	37.91
SmoothGrad (abs)	0.0879	0.5011	0.4133	38.00
Grad-CAM	0.1133	0.5464	0.4331	0.37
Smooth Grad-CAM	0.1372	0.5080	0.3708	18.37

- Every method beats Random on Insertion by a clear margin. SmoothGrad (mean) has the best Deletion (0.060).
- Grad-CAM's Deletion (0.113) is indistinguishable from Random (0.112). A likely but untested reason is that its 7x7 map removes a coarse blob and leaves much of the object visible.
- Grad-CAM and SmoothGrad (abs) differ by only 0.02 on Ins − Del, which is small against the per-image spread, so we treat them as comparable.
- Smooth Grad-CAM does not improve on plain Grad-CAM (0.371 vs 0.433) and costs about 50 times more time.

Ablations (30 images each, so only the larger trends are reliable):

- **Noise level:** with no noise SmoothGrad reduces to the vanilla gradient (Ins − Del 0.227). A flat optimum sits at about 0.05 to 0.2 (0.357 to 0.367), and noise of 0.5 drops it back to 0.248.
- **Sample count:** Ins − Del rises from 0.307 (N=1) to 0.360 (N=50), with diminishing returns beyond about 25 samples while cost grows linearly.

- **Grad-CAM layer:** `layer4` is clearly best (Ins − Del 0.470). `layer1` (0.091) is no better than Random (0.104), because deeper layers are more class-specific.

6. Challenges faced

Most problems were small API and robustness bugs found while testing, plus one practical problem (speed) and some difficulties in judging the results.

Grad-CAM

- Calling `GradCAM(...)(x, 5)` with an integer class crashed with `TypeError: 'int' object is not subscriptable`.
- Pointing the target at a layer that is not in the forward pass crashed with `TypeError: 'NoneType' object is not iterable`.
- The forward hook stayed attached to the model unless `remove()` was called.
- On Windows, `run_demo.py` ended with `UnicodeEncodeError` after the whole evaluation had finished.
- The 98 MB ResNet-50 weights download was interrupted when a run was cancelled.

SmoothGrad

- With `target=None`, Smooth Grad-CAM re-picked the predicted class on every noisy copy, so different copies could explain different classes.
- `smoothgrad` accepted only tensor targets, and bad arguments gave obscure errors (a `KeyError` for an unknown mode, a zero map or division error for `n_samples=0`).
- All three mode terms were computed on every iteration, the model was left in `eval()` after the call, and results could not be reproduced.
- A few extreme pixels made the maps look almost entirely dark.

Speed and evaluation

- SmoothGrad needs many forward and backward passes, and Deletion/Insertion needs many forward passes per image. A 50-image CPU run took roughly an hour.
- There is no ground-truth explanation, so we had to rely on proxy metrics. Our early runs used few images (20 to 64), with large per-image spread.
- Grad-CAM's Deletion score was no better than Random, and class discrimination was only partial (Pearson r = 0.54 and 0.80 between maps of different classes for SmoothGrad).

7. How we solved them

Each bug was fixed in the code and, where possible, locked in with a unit test, so it cannot return unnoticed.

Problem	Cause	Fix
Integer target crashed Grad-CAM	`target` was assumed to be a tensor	`target` is converted with `torch.as_tensor`, and SmoothGrad shares a `_resolve_target` helper, so int, list and tensor targets all work and agree
Unused layer gave an obscure `TypeError`	The hook never fired, so no activations were stored	An explicit check raises a `RuntimeError` that says the target layer produced no output
Hook left on the model	No cleanup path	Added `__enter__` / `__exit__`, so `with GradCAM(model, layer) as g:` removes the hook automatically
Windows `UnicodeEncodeError`	`metrics.md` was written in cp1252, which cannot store the arrows in the table header	Write the file with `encoding="utf-8"`, or set `$env:PYTHONUTF8=1`
Interrupted weights download	The run was cancelled mid-download	Re-run and wait; the weights are cached in `~/.cache/torch/hub/checkpoints/`
Different classes per noisy copy	`target=None` was resolved inside the noise loop	The class is fixed once on the clean image
Obscure errors for bad arguments	No input validation	Clear `ValueError` messages for `n_samples < 1`, negative noise, unknown `mode` or `reduce`, and bad percentiles
Wasted compute, mode side effect, no reproducibility	All mode terms computed each step, model mode not restored, global RNG only	Only the requested term is computed, the previous train/eval mode is restored, and `seed=` uses a private `torch.Generator` that leaves the global RNG untouched

Problem	Cause	Fix
Dark maps	A few extreme pixels dominate the normalisation	Optional `clip_percentile` (for example 99) caps each map before normalising; pixel ranking and Deletion/Insertion are unaffected
Slow runs	Many passes per image on a CPU	Used small settings for quick checks (`--n_eval 20 --n_samples 10 --skip_ablation`), batched the class scores in one backward pass, and ran the final evaluation on a Kaggle T4: 200 images in about 12 minutes
Weak or noisy evidence	Few images, proxy metrics, partial class discrimination	Increased to 200 images for the final table, added Random as a baseline, and ran sanity checks (run-to-run agreement, class discrimination, cascading model randomisation)

The tests themselves give us confidence in the core code. For example, Grad-CAM on ResNet reduces exactly to CAM and matches a direct computation from the `fc` weights. For a linear model, SmoothGrad equals the vanilla gradient at any noise level, and the input gradient matches a numerical finite-difference derivative. The core SmoothGrad claim also checks out empirically. Maps from independent seeds agree more at N = 16 than at N = 1, and on real images run-to-run Pearson correlation rose from 0.22 to 0.55 and from 0.47 to 0.73 between N = 1 and N = 25.

8. Conclusion and references

Both methods work and beat a random baseline by a wide margin, but they trade off speed against pixel-level detail. Grad-CAM is fast and finds the right region, while SmoothGrad is slower but gives pixel-level maps and the best Deletion score. Smoothing did not help Grad-CAM.

The results have limits. The ablations use only 30 images, Deletion/Insertion are one view of faithfulness, and the sanity checks use single images and seeds, so small gaps should not be over-read.

References

- Selvaraju et al., *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*, ICCV 2017.
- Smilkov et al., *SmoothGrad: removing noise by adding noise*, 2017.

- Petsiuk et al., *RISE: Randomized Input Sampling for Explanation of Black-box Models*, BMVC 2018.
- Adebayo et al., *Sanity Checks for Saliency Maps*, NeurIPS 2018.